

Security Plan — Indoor Spatial Navigation Platform

Team: Mubassir Sapa (Project Lead / Full Stack), Hasan (Full Stack), Paschal Chidiutor Ibeh (Frontend) **System under review:** indoor_map — Next.js 16 + Payload CMS 3 application with a Payload admin panel, REST/GraphQL API, and a public map viewer / organization dashboard / drag-and-drop map editor.

Environments:

- **Development:** local SQLite (@payloadcms/db-sqlite) and local filesystem media storage — used for day-to-day development only.
- **Production:** Vercel (hosting, SSR + Partial Prerendering, TLS termination, edge network) + **Cloudflare** (domain/DNS, CDN in front of Vercel, object storage for uploaded media) + **MongoDB Atlas** (managed database).

This plan was produced by reading the current codebase (collections, access-control rules, auth flows, validation schemas, and CI config) rather than treating the system abstractly. Findings that reference specific files were confirmed against the code at the time of writing. Sections 4–6 distinguish dev-only concerns from the production Vercel/Cloudflare/Atlas stack where relevant.

1. Threat Identification

1.1 Assets

Asset	Description	Sensitivity
User accounts (users collection)	Email, password hash, role, organization link	High — identity & access
Organizations (organizations)	Tenant records owning floors/maps	Medium
Floors / MapNodes / MapObjects / PathEdges	The indoor map graph (rooms, paths, POIs) owners create	Medium — business data, integrity-critical
Media	Uploaded floor-plan images	Medium
PAYLOAD_SECRET, RESEND_API_KEY, DATABASE_URL (MongoDB Atlas connection string in production), Cloudflare API token / storage access keys	Server secrets (.env locally; Vercel encrypted environment variables in production)	Critical
Auth tokens (payload-token cookie, JWT)	Session material	High

1.2 Threat Table (STRIDE-based) with Risk Prioritization

Risk = Likelihood (1–3) × Impact (1–3).

#	Threat	STRIDE	Likelihood	Impact	Risk	Testing Focus
---	--------	--------	------------	--------	------	---------------

T1	Cross-tenant tampering: any authenticated user (any role/org) can create/update/delete another organization's floor/node/object/edge data, because <code>Floors/MapNodes/MapObjects/PathEdges</code> define no <code>access</code> rules and fall back to Payload's default check (any logged-in user, not scoped to the record's owner). Separately, the app's own dashboard/map-editor server actions bypass that check entirely via Payload's Local API <code>overrideAccess</code> behavior, with no manual ownership check in its place	Tampering	3	3	9 – Critical	Authorization testing (§3), API fuzzing
T2	Unrestricted file upload to <code>media</code> (no mime/size limits, open access)	Tampering / DoS	2	3	6 – High	Input validation, upload testing (§4)
T3	Credential stuffing / brute force on <code>/signin</code> (no rate limiting observed)	Spoofing	3	2	6 – High	Auth testing (§3)
T4	Weak JWT signing secret if <code>PAYLOAD_SECRET</code> env var is unset (falls back to <code>""</code>)	Spoofing / Tampering	1	3	3 – Medium	Config/secret audit (§5)
T5	NoSQL injection against MongoDB Atlas in production (operator injection, e.g. <code>\$gt/\$ne/\$where</code> smuggled through a JSON body into a query filter)	Tampering	1	3	3 – Medium	Injection testing (§4) — mitigated by Payload's query builder + Zod input typing, verify no route ever passes raw <code>req.body/req.query</code> into a Mongo filter
T6	Stored XSS via Lexical rich-text fields or map labels rendered on public map pages	Tampering / Info Disclosure	2	2	4 – Medium	Injection testing (§4)
T7	Leftover example endpoint <code>/my-route</code> and GraphQL Playground reachable in production	Info Disclosure	1	1	1 – Low	Config review (§5/§6)
T8	Password-reset token leakage/replay (email link contains token in URL)	Info Disclosure	1	2	2 – Low	Auth testing (§3)

T9	Denial of Service via large drag-and-drop editor payloads (unbounded node/object counts per save)	DoS	1	2	2 – Low	Load/input-boundary testing
T10	MongoDB Atlas network access list misconfigured (e.g. left at <code>0.0.0.0/0</code>) — exposes the database directly to the internet if credentials leak	Tampering / Info Disclosure	1	3	3 – Medium	Infra config review (§5)
T11	Cloudflare SSL/TLS mode set to "Flexible" instead of "Full (strict)" — leaves the Cloudflare→Vercel hop unencrypted even though the client→Cloudflare hop is HTTPS	Info Disclosure	1	2	2 – Low/Medium	Infra config review (§5)
T12	Cloudflare storage bucket for media misconfigured (public write, or listable)	Tampering / Info Disclosure	1	2	2 – Low/Medium	Infra config review (§5)

Verified, not assumed, on T1: unauthenticated (anonymous) REST/GraphQL requests were tested directly against a running instance — `POST /api/floors`, `POST /api/media`, and an unauthenticated GraphQL `Floors` query all returned `403 "You are not allowed to perform this action."` Anonymous access is correctly blocked, because Payload's default access check (`defaultAccess = ({ req: { user } }) => Boolean(user)`, confirmed in `node_modules/payload/dist/auth/defaultAccess.js`) requires *some* authenticated session. The real gap is authenticated-but-unauthorized access: that same default has no concept of organization or role, so any signed-up user (trivial to obtain via public signup) passes it for every record, on every collection that doesn't override it. It gets worse for the app's own flows — `floorEditorActions.ts` calls `payload.create/update/delete` without setting `overrideAccess: false`, and Payload's Local API defaults `overrideAccess` to `true` (confirmed in `node_modules/payload/dist/collections/operations/local/{create,update,delete}.js`), which skips access-control evaluation entirely regardless of collection config. Combined with those same server actions (e.g. `toggleFloorStatusAction`, `updateMapObject`) accepting an arbitrary id from the client with no check that it belongs to the caller's organization, this is exploitable today by any two registered users in different organizations, not a theoretical default.

Note on DDoS vs. brute force (T3): these are two distinct threat classes handled differently in this plan. *Volumetric/network-layer DDoS* against the app is already given a baseline mitigation for free, as a side effect of the infrastructure choice — Cloudflare sits in front of Vercel and absorbs/filters large-scale traffic floods at the edge before they reach the origin, with no application code required. *Application-layer abuse* (e.g. scripted credential stuffing against `/signin`, one request at a time, indistinguishable from normal traffic by volume alone) is not covered by that and would need dedicated app-level rate limiting — which this project is consciously not building (see §6/§7, F3).

1.3 Risk → Testing Mapping

- **Critical/High risks (T1–T3)** drive the priority order of testing: authorization testing of every Payload collection first, then upload validation, then auth brute-force testing.
- **Medium risks (T4–T6)** are covered by configuration audits and injection testing.
- **Low risks (T7–T9)** are tracked but scheduled after the above are closed.

2. Testing Techniques and Tools

Technique	Tool(s)	How it's applied here
-----------	---------	-----------------------

SAST (Static Application Security Testing)	<code>eslint + eslint-plugin-security</code> (add to existing <code>eslint.config.mjs</code>), <code>npm audit</code> / GitHub Dependabot, Semgrep (<code>p/owasp-top-ten</code> ruleset) run against <code>src/</code>	Scans TypeScript source and dependencies for known-vulnerable packages and unsafe patterns (e.g. <code>dangerouslySetInnerHTML</code> , <code>unsanitized eval</code>) before merge
DAST (Dynamic Application Security Testing)	OWASP ZAP (baseline + active scan) against a local <code>next dev/next start</code> instance; Burp Suite (Community) for manual proxying of the <code>signin/signup/editor</code> flows	Exercises the running app the way an attacker would: <code>crawls /</code> , <code>/api/[...slug]</code> , <code>/api/graphql</code> , submits forms with attack payloads
Manual code review	Peer review checklist (this document, §3–§5) applied to every new Payload collection and server action	Human review of <code>access</code> blocks, Zod schemas, and server actions — this is how T1/T2 were found
Input validation checks	Zod schemas already in <code>src/validations/shared/index.ts</code> and <code>src/features/*/validations/*.ts</code> ; extend with server-side re-validation everywhere a server action currently trusts client data	Confirms both client and server enforce the same rules
Authentication/Authorization testing	Manual Postman/Burp requests directly against <code>/api/users</code> , <code>/api/floors</code> , <code>/api/map-nodes</code> , etc. with no/low-privilege/other-tenant tokens	Confirms Payload <code>access</code> rules actually block cross-tenant and anonymous access
Dependency/config scanning	<code>npm audit</code> , GitHub Actions extension of existing <code>.github/workflows/lint.yml/test.yml</code> to add an <code>audit/semgrep</code> job	Continuous check on every push/PR to <code>main</code> , <code>prev</code> , <code>dev</code>

3. Authentication and Authorization Testing

3.1 Current mechanism (as implemented)

- Payload's built-in local auth on the `users` collection (`src/collections/Users.ts`): email/password, email verification required, forgot/reset-password flow, JWT stored in an `httpOnly payload-token` cookie (`sameSite: "Lax"`, secure tied to `NODE_ENV === "production"`), 30-day token expiration.
- Route gating in `src/proxy.ts` (Next middleware): redirects anonymous users away from `/dashboard` and `/editor`, and redirects already-authenticated users away from auth pages. **Note:** this middleware only checks for the *presence* of the `payload-token` cookie for `isPrivateRoute` and calls `/api/users/me` to confirm identity for the auth-page redirect — it is a UX convenience, not the authorization boundary. The real authorization boundary is each collection's `access` config, which is why §3.3 matters.
- Roles: `admin` / `user` (`src/collections/constants/roles.ts`), enforced via `isAdmin` / `isAdminOrSelf` (`src/collections/access/index.ts`).

3.2 Password entropy check

Signup policy (`src/validations/shared/index.ts`): minimum 8 characters, must include lowercase, uppercase, digit, and special character.

An eight-character password randomly generated from the full ~94-character printable ASCII alphabet has a **theoretical maximum search space**:

Entropy = $\log_2(94^8) = 8 \times \log_2(94) \approx 8 \times 6.55 \approx 52.4 \text{ bits} \rightarrow \text{"Weak"} \text{ (36–59 bits)}$

This is only the theoretical search space when all eight characters are independently and uniformly random — it does **not** prove a real, user-chosen password actually has 52.4 bits of entropy. Real user-selected passwords (dictionary words with substitutions, keyboard patterns, personal dates) usually have considerably lower *actual* entropy than this ceiling, since humans don't pick characters uniformly at random. The 52.4-bit figure is a useful upper bound for comparing the policy's minimum requirement against the rubric's entropy scale, not a measurement of real password strength.

Finding: the policy is enforced correctly, but its *minimum allowed* length only reaches a ~52-bit theoretical ceiling, which the rubric's entropy scale classifies as Weak, not Strong (60+ bits) — and real user passwords at that length will typically sit below even that ceiling.

Recommendation: raise the minimum length to 10 characters, which raises the theoretical ceiling to Strong:

$\log_2(94^{10}) = 10 \times 6.55 \approx 65.5 \text{ bits} \rightarrow \text{"Strong"}$

This is a one-line change to `SIGNUP_CLIENT.VALIDATION_PASSWORD_MIN` / the `Fields.password` min in `src/validations/shared/index.ts`.

3.3 Access privilege testing (per collection)

Two different surfaces were tested, because they turned out to fail in different ways: (1) raw HTTP requests directly against `/api/<collection>` and `/api/graphql`, and (2) what access check the app's own server actions/adapters actually reach before touching the database.

(1) Raw REST/GraphQL — confirmed by direct testing against a running local instance:

Request		Result			
GET /api/floors (no auth)		403 Forbidden			
POST /api/floors (no auth)		403 Forbidden			
POST /api/media (no auth)		403 Forbidden			
GET /api/media (no auth)		200 OK — full docs list, by design (<code>read: () => true</code>)			
GraphQL query { Floors { docs { id name } } } (no auth)		403 Forbidden			

Collection	create	read	update	delete	Result
users	admin only	admin or self	admin or self	admin only	Pass — correctly scoped
organizations	admin only	any logged-in user	admin only	admin only	Pass — accepted by design: any authenticated user can read <i>all</i> organizations, not just their own. Not treated as a leak here: org <code>name/type</code> aren't confidential, and the product is a venue directory/navigation platform where seeing other participating organizations matches the intended experience. See §6 (F5).
media	any logged-in user (confirmed: anon POST → 403)	anyone (by design)	any logged-in user, not org-scoped	any logged-in user, not org-scoped	Partial fail — anonymous create/update/delete is correctly blocked, but any authenticated user (any org) can create/update/delete any media record, and there's no file type/size restriction

<code>floors.map-nodes.map-objects.path-edges</code>	any logged-in user (confirmed: anon POST → 403)	any logged-in user (confirmed: anon GET → 403)	any logged-in user, not org-scoped	any logged-in user, not org-scoped	Fail — anonymous access is blocked, but no <code>access</code> block is defined in <code>Floors.ts</code> , <code>MapNodes.ts</code> , <code>MapObjects.ts</code> , <code>PathEdges.ts</code> , so Payload's default (<code>Boolean(user)</code>) lets <i>any</i> authenticated user, regardless of organization or role, read/write/delete <i>any</i> record. (The "other-org token" case wasn't independently re-tested with two live accounts in this pass — this follows directly from the code: the default check does no ownership comparison, and no collection here overrides it.)
--	---	--	------------------------------------	------------------------------------	---

(2) App server actions / Local API — confirmed by code review:

Path	What actually gates it
Floors — <code>createFloorAction / toggleFloorStatusAction</code> → <code>dashboard-pl.adapter.ts</code>	<code>payload.create/payload.update</code> called with <code>overrideAccess: true</code> explicitly — Payload access control is skipped entirely. The action itself only checks <code>getCurrentUser()</code> (any logged-in user); <code>toggleFloorStatusAction(floorId, publish)</code> never checks that <code>floorId</code> belongs to the caller's organization
MapObjects / MapNodes / PathEdges — <code>floorEditorActions.ts</code> (create/update/delete)	<code>overrideAccess</code> isn't set → defaults to <code>true</code> (Payload's Local API default, confirmed in <code>node_modules/payload/dist/collections/operations/local/{create,update,delete}.js</code>) — same bypass, with no ownership check in the action code either
Media — <code>uploadFloorReferenceImage</code>	Same pattern: explicit <code>overrideAccess: true</code>

This means adding `access` blocks to the collections alone (the original F1 fix) would **not** close the hole for the app's own dashboard/editor flows — those calls skip collection access regardless of what the collection declares, because of `overrideAccess`. The fix has to touch both layers (see §6, F1).

This is the single most important authorization gap in the system and is the top item in the mitigation plan (§6).

4. Input Validation and Injection Testing

Vector	Where it applies	Client-side check	Server-side check	Test case
SQL/NoSQL Injection	Dev: SQLite via <code>@payloadcms/db-sqlite</code> . Prod: MongoDB Atlas via <code>@payloadcms/db-mongodb</code>	n/a	Payload's query builder + Zod schemas type-check input before it reaches the database; confirm no route ever spreads raw <code>req.body/req.query</code> directly into a <code>find/update</code> filter	Dev: submit ' OR '1'='1,1; DROP TABLE users;-- in text fields. Prod: submit <code>{"email": {"\$gt": ""}, "password": {"\$gt": ""}}</code> as a signin JSON body and ? <code>search[\$where]=...</code> -style query-string operators; confirm rejected/ignored rather than executed as a query operator

Cross-Site Scripting (XSS)	Lexical rich-text fields, map node/object <code>label</code> fields rendered in <code>MapViewSvg.tsx</code> / editor panels	Zod string schemas (no HTML stripping)	Payload/Lexical serializes rich text as structured JSON, not raw HTML, reducing risk	Submit <code><script>alert(1)</script></code> and <code></code> as a floor name, node label, and org name; verify it renders as inert text everywhere it's displayed (dashboard, public map, editor, email templates)
Command Injection	None identified — no shell/child-process execution in the reviewed code	n/a	n/a	Re-check whenever a feature (e.g. future AI blueprint import) shells out to an external tool
Mass assignment / over-posting	Server actions (<code>signup.ts</code> , <code>create-floor.ts</code> , etc.) that pass validated data into Payload's <code>create/update</code>	<code>SignupActionSchema</code> , <code>CreateFloorSchema</code> (Zod) restrict fields	Confirm the action never spreads raw <code>req.body</code> into Payload calls, only the validated object	Attempt to POST role: <code>"admin"</code> or <code>organization: <other-org-id></code> in the <code>signup/floor-create</code> payload and confirm it's rejected/ignored
File upload validation	<code>media</code> collection (<code>upload: true</code> , no <code>mimeType</code> s/size limit configured)	Browser <code>accept</code> attribute only (bypassable)	None found	Upload a <code>.php/.html/.svg</code> (script-bearing) file and an oversized file; confirm it is rejected once <code>mimeType</code> s + size limits are added (see §6)

5. Data Security and Privacy Checks

- **Transport security:** Vercel automatically terminates TLS for the app, and the `secure` cookie flag is already conditioned on `NODE_ENV === "production"` (`src/collections/Users.ts`). Because Cloudflare sits in front of Vercel as a CDN, there are **two** hops to verify, not one: client→Cloudflare (covered by Cloudflare's own edge certificate) and Cloudflare→Vercel origin. Cloudflare's SSL/TLS mode must be set to **Full (strict)** — "Flexible" mode would leave the Cloudflare→origin hop in plain HTTP even though the browser shows a padlock (T11). Also enable HSTS at the Cloudflare edge.
- **Encryption at rest:** MongoDB Atlas encrypts data at rest by default (AES-256, transparent — no application code required), and Cloudflare's object storage does the same for uploaded media. This means encryption-at-rest for production is satisfied by the managed platforms themselves rather than something the app needs to implement. The application stores limited personal and account data — email addresses, roles, organization associations, and authentication/reset tokens, alongside the map/floor content. Of these, the password is the one field stored as a hash rather than plaintext, additionally protected by Payload's auth module regardless of the underlying storage; the rest (email, role, org association, tokens) rely on Atlas's at-rest encryption and TLS-in-transit rather than app-level hashing, since they need to be readable by the app. (In the local dev environment, the SQLite file is not encrypted — acceptable since dev data is disposable and never holds real user data.)
- **Secrets management:** `PAYLOAD_SECRET`, `RESEND_API_KEY`, `DATABASE_URL` (now a MongoDB Atlas connection string containing DB credentials), and the Cloudflare API token/storage access keys must be stored as **Vercel encrypted environment variables**, scoped separately per environment (Production vs. Preview) so preview deployments never receive production database credentials. Locally these stay in the gitignored `.env`. Separately, `src/payload.config.ts` currently does `process.env.PAYLOAD_SECRET || ""` and `process.env.DATABASE_URL || ""` — if a variable is missing in any environment, the app **silently starts with an empty secret/connection string** instead of failing. This should fail securely (§6).
- **Data leakage / exposure:**
 - `src/app/my-route/route.ts` is unremoved Payload boilerplate returning a static JSON message on an unauthenticated route — low risk but unnecessary attack surface and should be deleted.
 - `/api/graphql-playground` (`src/app/(payload)/api/graphql-playground/route.ts`) exposes the GraphQL schema/introspection UI; confirm Payload's production guard actually disables it when deployed (verify by hitting the route on the deployed build, not just locally).
 - Error responses (`errorResponse` in `src/lib/responses/app-response.ts`) should be checked to ensure they never forward raw database/Payload error messages to the client (fail securely — generic message to user, detailed log server-side).
 - **MongoDB Atlas network access list** must be restricted (ideally via Atlas's Vercel integration, which manages IP allowlisting automatically) rather than left open to `0.0.0.0/0` (T10).
 - **Cloudflare storage bucket** for media must only allow the app's server to write; public access should be read-only for files actually intended to be public, and bucket listing should be disabled (T12).
 - Cloudflare CDN caching must not cache authenticated/private API responses — confirm `Cache-Control` headers on `/api/*` and dashboard/editor routes prevent private data from being cached at the edge and served to a different user.
- **Third-party data flow:** password-reset/verification emails go through Resend (`RESEND_API_KEY`); confirm the Resend account only sends from a verified domain (SPF/DKIM configured) to avoid the platform being used for phishing/spoofing. Cloudflare and Vercel are additional processors of user traffic/DNS in this stack; the application stores limited personal and account data (email addresses, roles, organization

associations, authentication/reset tokens — with only the password stored as a hash rather than plaintext), so their standard compliance posture is judged sufficient rather than requiring a dedicated data-processing agreement review at this project's scale.

6. Reporting and Risk Mitigation Plan

ID	Finding	Severity	Proposed Fix	Owner	Re-test Plan
F1	Two-layer authorization gap on map data: (a) <code>Floors/MapNodes/MapObjects/PathEdges</code> define no <code>access</code> rules, so any authenticated user (any org/role) can read/write any organization's records via raw REST/GraphQL — anonymous access is correctly blocked (verified, see §3.3); (b) the app's own server actions (<code>floorEditorActions.ts</code> , <code>dashboard-pl.adapter.ts</code>) bypass access control entirely via Payload's Local API <code>overrideAccess</code> (explicit <code>true</code> in some calls, default <code>true</code> in others), with no manual organization-ownership check in its place	Critical	Two changes needed together: (1) add <code>access</code> blocks to all four collections scoping create/update/delete to the record's owning organization (mirroring <code>Organizations.ts</code>); (2) update the server actions/adapters to pass <code>overrideAccess: false</code> plus the real <code>req/user</code> , or add an explicit "does this record belong to the caller's org" check before every mutation	Backend dev	Re-run §3.3: (a) direct REST/GraphQL calls as a user from a different org must return 403; (b) calling <code>toggleFloorStatusAction/updateMapObject</code> etc. with another org's id must be rejected, not silently succeed
F9	Encryption at rest for production data	Satisfied by platform (verify only)	MongoDB Atlas and Cloudflare storage both encrypt at rest by default — no app-level work needed. Local dev SQLite remains unencrypted, which is an accepted risk since dev data is disposable.	N/A	Confirm Atlas cluster shows encryption-at-rest enabled (default) in its dashboard; revisit if the schema later stores genuinely sensitive personal data
F10	MongoDB Atlas network access list open to <code>0.0.0.0/0</code>	Medium — accepted risk (no fix planned)	None planned. Vercel serverless functions don't have static outbound IPs, so strict IP allowlisting isn't practical without the paid Atlas–Vercel integration; the actual protection is the Atlas username/password (stored as a Vercel encrypted env var) plus Atlas's enforced TLS-only connections, both of which hold regardless of the network access list setting	N/A	Re-check if the project adopts the Atlas–Vercel network integration or a fixed egress IP add-on later

F11	Cloudflare SSL/TLS mode not yet confirmed as "Full (strict)"	Medium	Set Cloudflare SSL/TLS encryption mode to Full (strict); enable HSTS	Backend dev / DevOps	Inspect the Cloudflare→Vercel request with <code>curl -v</code> or Cloudflare's own SSL diagnostic; confirm origin connection is HTTPS, not HTTP
F12	Cloudflare storage bucket permissions not yet confirmed	Low/Medium	Restrict write access to the app's server credentials only; disable public bucket listing; public read only for intentionally public media	Backend dev	Attempt an unauthenticated write/list against the bucket URL directly; expect denial
F2	Media collection: read is intentionally public (confirmed, GET <code>/api/media</code> → 200); create/update/delete correctly require <i>some</i> authenticated user (confirmed, anon POST <code>/api/media</code> → 403) but aren't scoped to org/role, and there's no file type/size limit on <code>upload: true</code>	High	Add explicit <code>access.isLoggedInIn</code> (or org-scoped) for create/update/delete for clarity/defense-in-depth, and add <code>upload: { mimeTypes: ["image/png", "image/jpeg", "image/webp"], },</code> with a size limit	Backend dev	Repeat upload test as an authenticated user with disallowed mime types/oversized files; expect rejection
F3	No logout on repeated failed <code>/signin</code> attempts, and no throttling on <code>/forgot-password</code> ; no dedicated DDoS mitigation beyond Cloudflare's default	High — partially fixed	Fix (cheap, built-in): add <code>auth: { maxLoginAttempts: 5, lockTime: 10 * 60 * 1000 }</code> to <code>Users.ts</code> — Payload enforces account logout natively, confirmed supported in its auth types, no Redis/Upstash required. Accepted risk (no fix planned): <code>/forgot-password</code> throttling and broader volumetric DDoS mitigation beyond Cloudflare's default edge protection — Payload has no built-in equivalent for arbitrary endpoints, and custom throttling is scoped as future work beyond this capstone (see §1.2 note on T3)	Backend dev	Fixed part: scripted 6-attempt burst against <code>/signin</code> with a valid email; expect the account locked after the 5th attempt regardless of correct password. Accepted part: not tested — no fix planned
F4	<code>PAYLOAD_SECRET/DATABASE_URL</code> silently default to "" if unset	Medium	Replace <code>process.env.X ""</code> in <code>payload.config.ts</code> with the existing <code>requireEnv()</code> helper (<code>src/lib/env.ts</code>) so startup fails loudly	Backend dev	Unset the var locally and confirm the app refuses to boot instead of starting insecurely

F5	Cross-tenant read on organizations collection (any authenticated user can read all orgs)	Medium — accepted risk (no fix planned)	None planned. name/type are the only fields exposed and neither is confidential; the product is a venue directory/navigation platform, so a logged-in user seeing the list of participating organizations matches the intended product experience rather than leaking anything. Would be revisited if the organizations collection ever grows fields with real sensitivity (billing info, internal contacts, etc.)	N/A	Re-check this justification whenever new fields are added to Organizations.ts
F6	Minimum password length (8) yields ~52-bit entropy ("Weak")	Low	Raise minimum to 10 characters (~65 bits, "Strong") in signup validation	Frontend/backend dev	Recompute entropy per §3.2; confirm new min is enforced client + server side
F7	Leftover /my--route boilerplate endpoint	Low	Delete src/app/my--route/route.ts	Any dev	Confirm route returns 404 after removal
F8	GraphQL Playground reachability in production not verified	Low	Confirm/deploy-test that Payload disables the playground when NODE_ENV=production ; if not, gate the route explicitly	Backend dev	Hit /api/graphql-playground on the deployed build and confirm it is unavailable

Closure process: each finding above becomes a tracked issue; once a fix lands, the listed re-test is re-run and the issue is only closed when the re-test passes. F1, F2, F4, and F3's login-lockout fix are blockers for any public/production deployment and should be resolved before end of Week 10. F3's **/forgot-password/DDoS** portion, F5, F9 (verify only), and F10 are accepted risks with no fix planned (see §7) — they are tracked here, not treated as blockers. F6–F8 should be resolved before the next milestone demo.

7. Remediation Roadmap

This roadmap sequences the fixes from §6 by severity and dependency so the highest-risk gaps close first. It will be updated as fixes land and re-tests pass.

Priority	ID	Fix	Target	Status
1	F1	Add access blocks to Floors , MapNodes , MapObjects , PathEdges scoping create/update/delete to the owning organization's admin/user	Before Week 10 submission	Planned
2	F2	Restrict Media create/update/delete to logged-in users; add mimeType s allowlist and a max upload size	Before Week 10 submission	Planned

3	F4	Replace <code>process.env.PAYLOAD_SECRET "" / DATABASE_URL ""</code> in <code>payload.config.ts</code> with the existing <code>requireEnv()</code> helper so misconfigured deploys fail at boot instead of running with a blank secret	Before Week 10 submission	Planned
—	F5	Scope <code>Organizations</code> read access to the requesting user's own org	N/A	Accepted risk — no fix planned (see §6/§7 justification)
4	F3	Add <code>maxLoginAttempts/lockTime</code> to <code>Users.ts</code> auth config (login-lockout portion only)	Before Week 10 submission	Planned. <code>/forgot-password</code> throttling and broader DDoS mitigation remain accepted risk — no fix planned
6	F6	Raise minimum password length from 8 to 10 characters in <code>Fields.password</code>	Next sprint (post-submission)	Deferred — low severity, no user-facing urgency
7	F7	Delete leftover <code>src/app/my-route/route.ts</code>	Before Week 10 submission	Planned — trivial cleanup
8	F8	Verify GraphQL Playground is disabled on the production build; gate explicitly if not	Before Week 10 submission	Planned — verification only, no code change expected
—	F10	MongoDB Atlas network access list open to <code>0.0.0.0/0</code>	N/A	Accepted risk — no fix planned (see §6 justification)
9	F11	Set Cloudflare SSL/TLS mode to Full (strict) + HSTS	Before production deployment	Planned
10	F12	Confirm Cloudflare storage bucket write/list permissions	Before production deployment	Planned
—	F9	Encryption at rest	N/A	Already satisfied by Atlas/Cloudflare defaults — verify only, no fix needed

Accepted risks:

- **F3 (rate limiting / DDoS)** is split: the `/signin` lockout piece is now a planned fix, not an accepted risk, since Payload's built-in `maxLoginAttempts/lockTime` makes it essentially free to add. What remains permanently accepted is `/forgot-password` throttling and dedicated DDoS mitigation beyond Cloudflare's default edge protection — Payload has no built-in equivalent for arbitrary endpoints, volumetric attacks already get a free baseline mitigation from the Cloudflare-in-front-of-Vercel architecture, and building custom throttling for `/forgot-password` doesn't block a safe demo of a student project with no real user base at risk. If the project continued past the capstone, this would be the first thing added.
- **F5 (cross-tenant read on organizations)** is accepted — `name/type` aren't confidential and showing all participating organizations to logged-in users matches the product's directory/navigation concept rather than leaking anything.
- **F10 (Atlas network access list open to 0.0.0.0/0)** is accepted — Vercel's serverless functions have no static outbound IP, so strict allowlisting isn't practical without paid tooling; the real protection (Atlas credentials + enforced TLS) holds regardless of this setting.
- **F6 (password minimum length)** is deferred, not accepted — it's a one-line fix with no infra dependency, tracked for the next sprint since it's cheap to close later but not urgent enough to block this submission.

All are tracked here so none are silently forgotten, with re-test/revisit criteria defined in §6 for if/when they're picked up.

Re-testing cadence: every fix above is re-tested using the specific procedure listed in the "Re-test Plan" column of §6 before its status is changed from "Planned"/"Deferred" to "Closed". The existing `.github/workflows/test.yml` and `lint.yml` CI jobs will gate merges of these fixes; a `security-audit` job (`npm audit` + Semgrep, per §2) will be added to catch regressions going forward.

References

- OWASP Foundation. *Web Security Testing Guide (WSTG)*. <https://owasp.org/www-project-web-security-testing-guide/stable/>
- NIST SP 800-12 Rev. 1, *An Introduction to Information Security*.
- NIST SP 800-218, *Secure Software Development Framework (SSDF)*.
- Seneca SE — *Software Security Overview* course notes.